Lab 7. ANN with PyTorch and model inferencing with ONNX runtime

Objective

- To implement and demonstrate neural network models by using **PyTorch**.

- To install PyTorch in your host machine and use it as the **ML framework** to create models instead of creating it from scratch with numpy that you had learnt from the previous labs.

- To install **ONNX** to export the model into the standard format which can be used in other frameworks and can be deployed (Inference) with different types of hardware (CPU, GPU, NPU etc).

Experiment 1 : PyTorch Installation

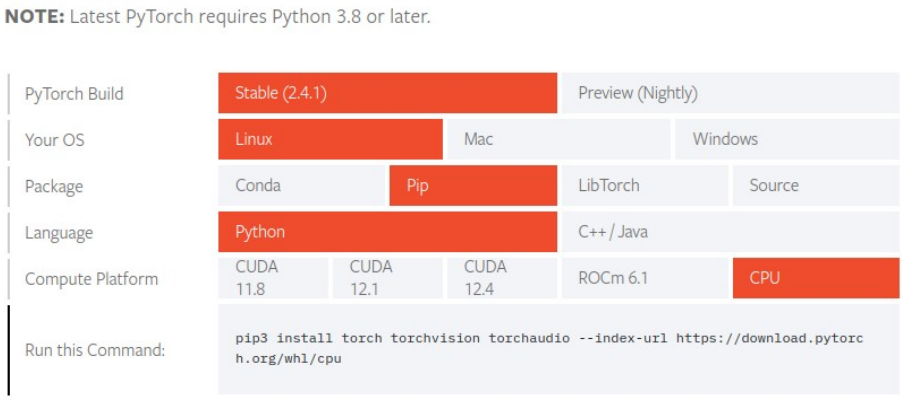
Step 1. Download & Installation

Go to this URL to get the installation command, based on your host machine's preferences: https://pytorch.org/get-started/locally/

Select the OS of your machine (**Windows, Mac, Linux**).

Python and **PIP** will be used as the language and the package installer for pytorch. You can activate your python environment as you prefer.

For the Compute Platform, select **CPU** to reduce the GPU setup problem (CUDA, CUDNN, ROCm etc) during the lab.



Windows
$ pip install torch torchvision torchaudio
Mac
$ pip install torch torchvision torchaudio
Linux
$ pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu

Warning

Please ensure that you have met the prerequisites depending on your package manager (We use PIP as our current package manager). **Python version >= 3.8** should be enough.

Step 2. Verify the installation

$ pip show torch

```
Name: torch
Version: 2.12.0
Summary: Tensors and Dynamic neural networks in Python with strong GPU acceleration
Home-page:
Author:
Author-email: PyTorch Team <packages@pytorch.org>
License: BSD-3-Clause
```

After installation is completed. Do the experiment on PyTorch's tensor data with this python code:

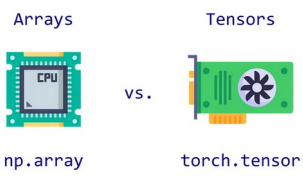
```python
import torch

# generate randomized tensor
rand = torch.rand(3, 4)
print(rand)

# basic tensor multiplication
data1 = [[1, 2], [3, 4]]
data2 = [[1], [2]]
data3 = [1, 2]
t1 = torch.tensor(data1)
t2 = torch.tensor(data2)
t3 = torch.tensor(data3)
r1 = torch.matmul(t1, t2)
r2 = torch.matmul(t1, t3)
print("matrix A =", t1)
print("vector x =", t2)
print("transpose of x =", t3)
print("A * x =")
print(r1)
print("xT * A =")
print(r2)
```

A **PyTorch Tensor** data-type is basically the same as a **numpy array**: it is just a generic n-dimensional array to be used for arbitrary numeric computation. We can do vector or matrix operations in linear algebra to the tensor data.

The biggest difference between a numpy array and a PyTorch Tensor is that a **PyTorch Tensor can run on either CPU or GPU.** There are the Vector/Matrices operation which can be **faster** with GPU or AI acceleration hardware. Learn more information by reading at this article: https://www.dailydoseofds.com/sklearn-models-are-not-deployment-friendly-supercharge-them-with-gpus-first/



Experiment 2 : ANN for Iris dataset using PyTorch

Step 1. Load Iris dataset and convert into PyTorch's tensor data type.

```
# Load the Iris dataset from local .csv file
# dataset can be founded in here : https://gist.github.com/netj/8836201
import pandas as pd
import numpy as np

filename = 'iris.csv'
df = pd.read_csv(filename)

# Create feature set : X by drop label column in dataframe
X=df.drop('variety', axis=1)
X = X.values

# Label the output classes
# Since Setosa and Versicolor are linearly separable and between Setosa and Virginica too,
# We will do the binary classification between Setosa (0) and group of Versicolor + Virginica (1)
y=df['variety']
y = y.values.reshape(150, 1)

l_y = []
for i in range(len(y)):
    if y[i, 0] == 'Setosa':
        l_y.append([1, 0, 0])
    elif y[i, 0] == 'Versicolor':
        l_y.append([0, 1, 0])
    elif y[i, 0] == 'Virginica':
        l_y.append([0, 0, 1])
y = np.array(l_y)

# convert data into pytorch tensor
import torch
X = torch.Tensor(X)
y = torch.Tensor(y)
print(X)
print(y)
print(X.shape)
print(y.shape)
```

Step 2. Split the data into a training set and testing set.

```
# Prepare the data
# Split the data into training set and testing set
from sklearn.model_selection import train_test_split
from torch.utils.data import TensorDataset, DataLoader

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=23)

# pack the feature X and output y together
train_ds = TensorDataset(X_train, y_train)

# create the training data, batch size.
batch_size = 1
train_dl = DataLoader(train_ds, batch_size, shuffle=True)
```

Step 3. Create ANN model for **Iris classifier**. We choose the **4-8-3 structure** as the default.

```
# Create NN model using PyTorch
import torch
from torch import nn

class MLP(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.layer1 = nn.Linear(input_size, hidden_size)
        self.layer_out = nn.Linear(hidden_size, output_size)
        self.activation_fn = nn.Sigmoid()

    def forward(self, x):
        z1 = self.layer1(x)
        a1 = self.activation_fn(z1)
        z_out = self.layer_out(a1)
        y = nn.Softmax(dim=1)(z_out)
        return y

model_iris = MLP(4, 8, 3)
print(model_iris)
```

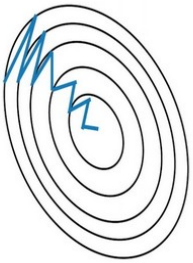
```
MLP(
  (layer1): Linear(in_features=4, out_features=8, bias=True)
  (layer_out): Linear(in_features=8, out_features=3, bias=True)
  (activation_fn): Sigmoid()
)
```

Step 4. Train the model.

The basic training algorithm (or Optimizer) that adjust the model's parameters to reduce the loss, is **Stochastic Gradient Descent (SGD)**. The example for this lab will use SGD but you could try to use **Adaptive Moment Estimation (ADAM)** which can increase the training speed. Or using the momentum feature on SGD.



Stochastic Gradient Descent **withhout** Momentum



Stochastic Gradient Descent **with** Momentum

```python
# Training
from torch.optim import SGD

# hyperparameters
learning_rate = 0.002
momentum = 0.1

# Loss function
loss_fn = nn.MSELoss(reduction='mean')
# Stochastic Gradient Descent (SGD) as optimizer in training procedure
#
# v = momentum * v - lr * grad(w)
# w = w + v
optimizer = SGD(model_iris.parameters(), lr=learning_rate, momentum=momentum)

num_epochs = 1000
batch_size = 1

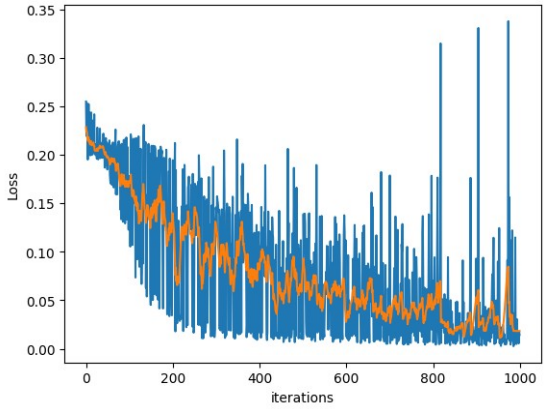
# Start iterate for model training
loss_arr = []
for epoch in range(num_epochs):
    for x_batch, y_batch in train_dl:
        # 1. Generate predictions
        predict = model_iris(x_batch)
        # 2. Calculate loss
        loss = loss_fn(predict, y_batch)
        # 3. Compute gradients
        loss.backward()
        # 4. Update parameters using gradients
        optimizer.step()
        # 5. Reset the gradients to zero
        optimizer.zero_grad()
    if epoch % (num_epochs/10) == 0:
        print(f'Epoch {epoch} Loss {loss.item():.4f}')
    loss_arr.append(loss.item())
```

```
Epoch 0 Loss 0.2550
Epoch 100 Loss 0.1654
Epoch 200 Loss 0.1945
Epoch 300 Loss 0.1419
Epoch 400 Loss 0.0165
Epoch 500 Loss 0.0099
Epoch 600 Loss 0.0171
Epoch 700 Loss 0.1787
Epoch 800 Loss 0.0066
Epoch 900 Loss 0.0166
```

Step 5. Plot the loss function vs epoch

```
# Plot the epoch vs loss
import matplotlib.pyplot as plt

# calculate moving average of loss
def wma(data, window_size):
    weights = np.arange(1, window_size + 1)
    return list(np.convolve(data, weights, mode='valid') / weights.sum())
loss_avg_arr = []
window_size=15
loss_avg_arr = wma(loss_arr, window_size)
for i in range(window_size - 1):
    loss_avg_arr.append(loss_avg_arr[-1])
plt.plot(list(range(num_epochs)), loss_arr)
plt.plot(list(range(num_epochs)), loss_avg_arr)
plt.xlabel('iterations')
plt.ylabel('Loss')
plt.show()
```



Step 6. Test the resulted model with test data

```
# Testing resulted model by measuring the accuracy
# accuracy = num of correction / num of total prediction

correct = 0
total = y_test.shape[0]
y_predict = model_iris.forward(X_test)
for i in range(total):
    # compare the predicted output with the actual output
    # by using the indices of the maximum value in the output layer
    val_predict, indices_predict = torch.max(y_predict[i, :], 0)
    val_actual, indices_actual = torch.max(y_test[i, :], 0)
    if indices_predict == indices_actual:
        correct += 1
print("number of correction=", correct, " out of ", total, ", accuracy=", correct/total)
```

```
number of correction= 30  out of  30 , accuracy= 1.0
```

Step 7. We can save the model in .pt file to be used later.

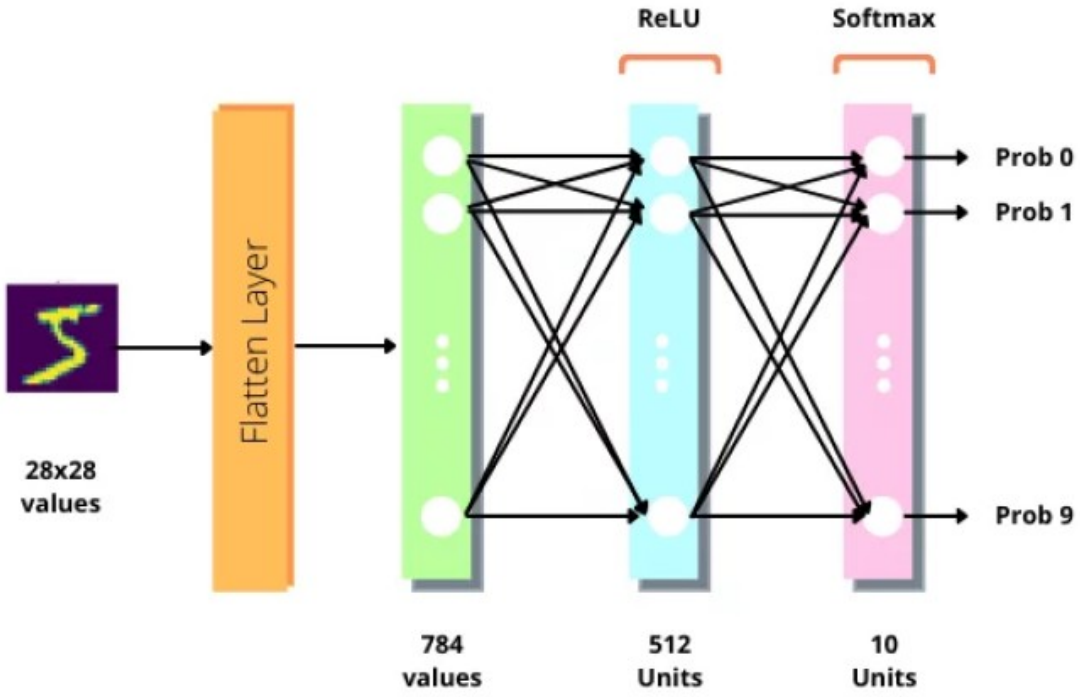
```
# Save the model in PyTorch .pth file with its current
path = 'model_iris.pth'
torch.save(model_iris, path)
```

Experiment 3 : ANN for MNIST handwritten digit using PyTorch

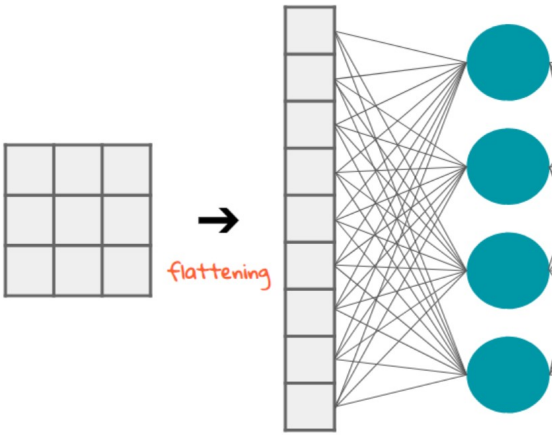
MNIST contains 70,000 images of handwritten digits: 60,000 for training and 10,000 for testing. The images are grayscale, 28x28 pixels, and centered to reduce preprocessing and get started quicker.



The configuration of ANN model for this problem could be looked like this:



Every pixel in a 28x28 image will be the input of ANN. We need to rearrange it by using the **Flattening** technique in the flatten layer first.



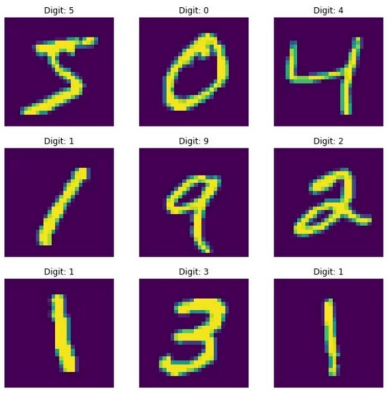
Please, look at these example .ipynb files in the MSTeam for how to read / manipulate the MNIST dataset.

resize_image.ipynb

load_mnist.ipynb

You need to create the classifier for the MNIST dataset with ANN by using the knowledge from Iris classifier in experiment 2. The structure is **784-100-100-10 (2 hidden layers, 10 outputs)** as the default. The size of the hidden layer can be adjusted only.

Pre-processing on the MNIST image for the experiment #3



Always preprocessing your data before moving it to train a neural network. Preprocessing the dataset makes it ready as input to the machine learning model. Images in our dataset are made up of grayscale pixels in range 0 – 255. Machine Learning models works better if the range of values dataset is using is small. So, **we convert its range to 0 – 1 by dividing it by 255.**

Note:

All MNIST image has a **black color for the background, white color for the digit area**, based on the grayscale shown in above image.

Step 1. Create ANN model for **MNIST classifier**. We choose the **784-100-100-10** structure as mentioned above.

[1]:
```
# Create NN model using PyTorch
import torch
from torch import nn

class MLP_MNIST(nn.Module):
    def __init__(self, input_size, h1_size, h2_size, output_size):
        super().__init__()
        self.flatten = nn.Flatten()
        self.layer1 = nn.Linear(input_size, h1_size)
        self.activation_fn1 = nn.Sigmoid()
        self.layer2 = nn.Linear(h1_size, h2_size)
        self.activation_fn2 = nn.Sigmoid()
        self.layer_out = nn.Linear(h2_size, output_size)
        self.softmax = nn.Softmax(dim=0)

    def flatten_mnist(self, x):
        result = self.flatten(x)[0]
        return result

    def forward(self, x):
        x = self.flatten_mnist(x)
        z1 = self.layer1(x)
        a1 = self.activation_fn1(z1)
        z2 = self.layer2(a1)
        a2 = self.activation_fn2(z2)
        z_out = self.layer_out(a2)
        y = self.softmax(z_out)
        return y

model_mnist = MLP_MNIST(784, 100, 100, 10)
print(model_mnist)
```

Step 2. Load & Prepare the **MNIST dataset**

```
import torch
import torchvision
from torchvision.datasets import MNIST
from torch.utils.data import TensorDataset, DataLoader

batch_size = 1
# Read the MNIST dataset by using torch and torchvision
# This will download the file to PATH if there's no dataset in the PATH
train_loader = DataLoader(
    MNIST('dataset', train=True, download=True,
          transform=torchvision.transforms.Compose([torchvision.transforms.ToTensor()])),
    batch_size=batch_size,
    shuffle=True
)

test_loader = DataLoader(
    MNIST('dataset', train=False, download=True,
          transform=torchvision.transforms.Compose([torchvision.transforms.ToTensor()])),
    batch_size=batch_size,
    shuffle=True
)

# Our label is integer (0-9). So, we need a function to convert it into 1x10 vector
# example 0 -> [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
# example 3 -> [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
def label_conversion_for_outputlayer(y):
    if y == 0:
        ret = [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
    elif y == 1:
        ret = [0, 1, 0, 0, 0, 0, 0, 0, 0, 0]
    elif y == 2:
        ret = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
    elif y == 3:
        ret = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
    elif y == 4:
        ret = [0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
    elif y == 5:
        ret = [0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
    elif y == 6:
        ret = [0, 0, 0, 0, 0, 0, 1, 0, 0, 0]
    elif y == 7:
        ret = [0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
    elif y == 8:
        ret = [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]
    elif y == 9:
        ret = [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
    return torch.Tensor(ret)
```

Step 3. Train the model.

```
# Training
from torch.optim import SGD

# hyperparameters
learning_rate = 0.01
momentum = 0.01

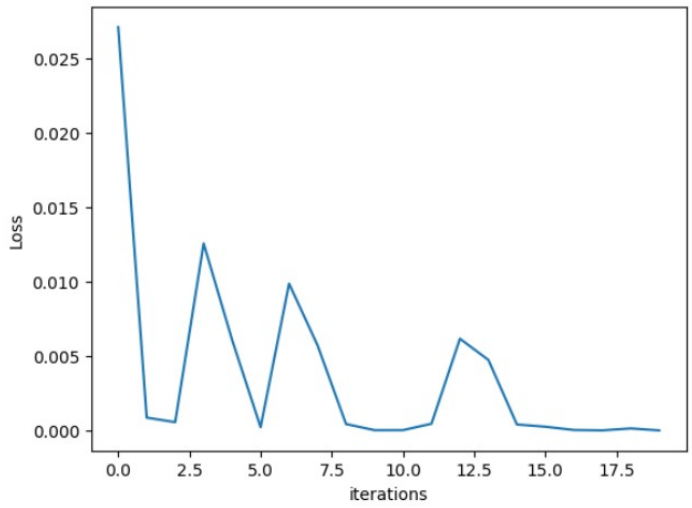
# Loss function
loss_fn = nn.MSELoss(reduction='mean')
# Stochastic Gradient Descent (SGD) as optimizer in training procedure
#
# v = momentum * v - lr * grad(w)
# w = w + v
optimizer = SGD(model_mnist.parameters(), lr=learning_rate, momentum=momentum)

num_epochs = 20
# Start iterate for model training
loss_arr = []
for epoch in range(num_epochs):
    for batch_idx, train_data in enumerate(train_loader):
        x_batch, y_batch = train_data
        # 0. Reset the gradients to zero
        optimizer.zero_grad()
        # 1. Generate predictions
        predict = model_mnist(x_batch)
        # 2. Calculate loss
        y = label_conversion_for_outputlayer(y_batch)
        loss = loss_fn(predict, y)
        # 3. Compute gradients
        loss.backward()
        # 4. Update parameters using gradients
        optimizer.step()
    print(f'Epoch {epoch} Loss {loss.item():.4f}')
    loss_arr.append(loss.item())
```

Step 4. Plot the loss function vs epoch

```
# Plot the epoch vs loss
import matplotlib.pyplot as plt

plt.plot(list(range(num_epochs)), loss_arr)
plt.xlabel('iterations')
plt.ylabel('Loss')
plt.show()
```



Step 5. Test the resulted model with our MNIST testing dataset & Measure the accuracy

```
# Testing resulted model by measuring the accuracy
# accuracy = num of correction / num of total prediction

correct = 0
total = 0
for idx, test_data in enumerate(test_loader):
    x, actual = test_data
    predict = model_mnist(x)
    actual = label_conversion_for_outputlayer(actual)
    val_predict, indices_predict = torch.max(predict, 0)
    val_actual, indices_actual = torch.max(actual, 0)
    if indices_predict == indices_actual:
        correct += 1
    total += 1
print("number of correction=", correct, " out of ", total, ", accuracy=", correct/total)
```

Step 6. Apply the resulted model to a new handwritten image **outside from MNIST**

```
# Test with your own handwritten images.
import cv2
import matplotlib.pyplot as plt

fig=plt.figure(figsize=(2, 5))
for i in range(1, 11):
    # Load image, Resize to 28x28, Invert and Normalize the pixel values
    path = 'img/' + str(i-1) + '.jpg'
    img_original = cv2.imread(path)
    img = cv2.resize(img_original, (28, 28), interpolation=cv2.INTER_AREA)
    img = cv2.bitwise_not(img)
    img = img / 256
    ax = fig.add_subplot(2, 5, i)
    ax.set_xticks([])
    ax.set_yticks([])
    plt.imshow(img)

    # Convert numpy image into tensor image
    # Select only 1 channel of RGB images is enough.
    # Add dimension to image to have the same format as MNIST data which has 4 dimensions with unsqueeze()
    img = torch.Tensor(img[:,:,0])
    img = img.unsqueeze(0).unsqueeze(0)

    # Prediction
    predict = model_mnist.forward(img)
    val_predict, indices_predict = torch.max(predict, 0)
    print(path, ", predicted number=", int(indices_predict))
    plt.title(str(int(indices_predict)))
```

```
img/0.jpg , predicted number= 0
img/1.jpg , predicted number= 1
img/2.jpg , predicted number= 8
img/3.jpg , predicted number= 3
img/4.jpg , predicted number= 4
img/5.jpg , predicted number= 0
img/6.jpg , predicted number= 6
img/7.jpg , predicted number= 7
img/8.jpg , predicted number= 8
img/9.jpg , predicted number= 7
```



Note, the code will use the handwritten image in img/ folder to test our MNIST model. Please download the images in the MSTeam's class material.

Step 7. Export the Pytorch model .pth/.pt. It will be used in experiment #5.

```
# Save the model in PyTorch .pth file with its current state
import torch
path = 'model_mnist.pth'
torch.save(model_mnist, path)
```

Experiment 4 : ONNX Installation

What is ONNX?

The **O**pen **N**eural **N**etwork e**X**change (ONNX) is an open format designed to represent any type of machine learning model. The model can be NN-based or non-NN-based (Decision Tree, SVM, Rule-based etc.) These models can be trained using various frameworks including PyTorch, Keras/TensorFlow, Scikit-learn etc. Each of these frameworks generates trained models in different formats making them non-portable for use.

ONNX format allows for framework interoperability by providing a **uniform format** that acts as an **intermediate** between machine learning frameworks. This interoperability allows trained models to be easily deployed in **different platforms** or run the model on **ONNX runtime.**

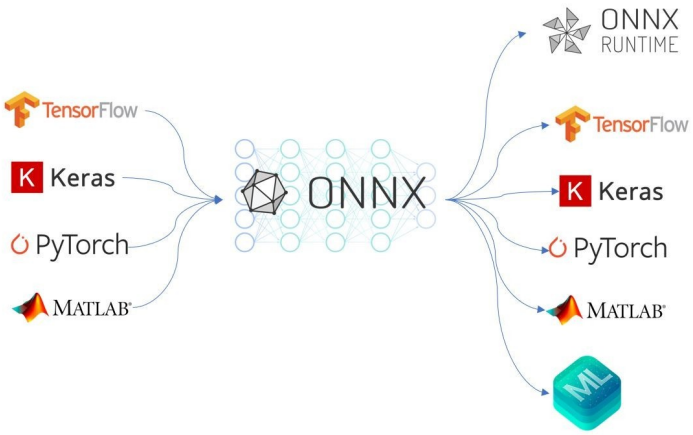


Fig. How ONNX is the intermediate between ML frameworks

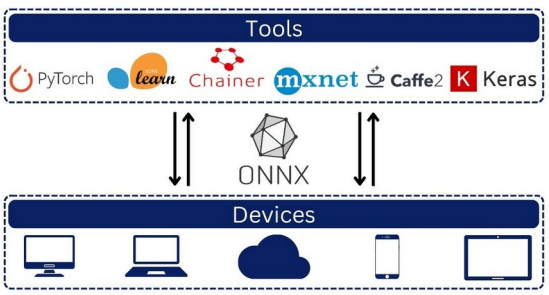


Fig. Deploying the ML model into the different device with ONNX

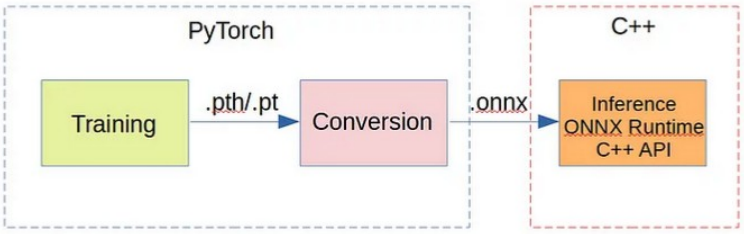


Fig. ONNX conversion process and deployment using ONNIX runtime

Step 1.

Install ONNX and its dependencies by using PIP.

$ pip install onnx

$ pip install onnxscript

Step 2.

Import the IRIS model from the experiment #2. Then, prepare the export configuration for the **ONNX format** file (.onnx) by using **torch.onnx.export()**.

(Change model_mnist.pth to model_iris.pth)

```
# Export ONNX formatted model from .pth model
import torch

path = 'model_mnist.pth'
loaded_model = torch.load(path, weights_only=False)
loaded_model.eval()
```

```
# Export ONNX formatted model from .pth model
import torch
input_size_iris = 4 # input for iris model
dummy_input = torch.rand(1, input_size_iris)
input_names = ["input"]
output_names = ["output"]
torch.onnx.export(loaded_model,
                  dummy_input,
                  "model_iris.onnx",
                  verbose=False,
                  input_names=input_names,
                  output_names=output_names,
                  export_params=True,
                  )
```

Then, save the .onnx file in the current directory by running onnx_program.save(). Change the filename in the parameter as **“model_iris.onnx”**

```
onnx_program.save("my_image_classifier.onnx")
```

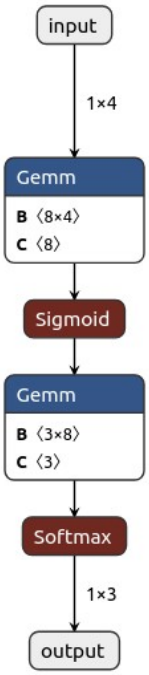
Step 3. Visualize the architecture of our exported ONNX model using **Netron.**

Netron can either be installed locally, or run directly from the web browser. Let's try the web version by opening the following link: https://netron.app/. It will ask the location of our .onnx file.

Check the model structure in the graph. It should be the same structure as the PyTorch model.



Example graph from our IRIS model (ANN 4-8-3)



Step 4. Load the ONNX file back into memory and check if it is well formed with the following code:

```
import onnx
onnx_model = onnx.load("my_image_classifier.onnx")
onnx.checker.check_model(onnx_model)
```

Change the filename in the parameter as your **"model_iris.onnx"** file

Step 5. Repeat step 3 and step 4. but with MNIST model.

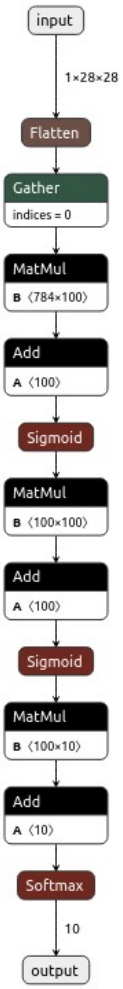
```
# Save the model in PyTorch .pth file with its current state
import torch
path = 'model_mnist.pth'
torch.save(model_mnist, path)
```

```
# Export ONNX formatted model from .pth model
import torch

path = 'model_mnist.pth'
loaded_model = torch.load(path, weights_only=False)
loaded_model.eval()

# input_size = 28*28
dummy_input = torch.rand(1, 28, 28)
input_names = ["input"]
output_names = ["output"]
torch.onnx.export(loaded_model,
                  dummy_input,
                  "model_mnist.onnx",
                  verbose=False,
                  input_names=input_names,
                  output_names=output_names,
                  export_params=True,
                  )
```

Example graph for MNIST model



Experiment 5: ONNX Runtime Installation & Inferencing

Step 1. Install ONNX Runtime

Install ONNX runtime on your PC to test our model by using PIP.

The reference for the installation :

https://onnxruntime.ai/docs/install/

https://onnxruntime.ai/docs/install/#install-onnx-runtime-cpu

Choose the installation for **only CPU mode**.

$ pip install onnxruntime

Step 2. Import the ONNX model

```
# Try to import the ONNX model from the previous section
import onnx

# Load the ONNX model
model = onnx.load("model_mnist.onnx")

# Check that the model is well formed
onnx.checker.check_model(model)

print(onnx.helper.printable_graph(model.graph))
```

```
graph main_graph (
  %input[FLOAT, 1x28x28]
) initializers (
  %layer1.bias[FLOAT, 100]
  %layer2.bias[FLOAT, 100]
  %layer_out.bias[FLOAT, 10]
  %onnx::MatMul_22[FLOAT, 784x100]
  %onnx::MatMul_23[FLOAT, 100x100]
  %onnx::MatMul_24[FLOAT, 100x10]
) {
  %/flatten/Flatten_output_0 = Flatten[axis = 1](%input)
  %/Constant_output_0 = Constant[value = <Scalar Tensor []>]()
  %/Gather_output_0 = Gather[axis = 0](%/flatten/Flatten_output_0, %/Constant_output_0)
  %/layer1/MatMul_output_0 = MatMul(%/Gather_output_0, %onnx::MatMul_22)
  %/layer1/Add_output_0 = Add(%layer1.bias, %/layer1/MatMul_output_0)
  %/activation_fn1/Sigmoid_output_0 = Sigmoid(%/layer1/Add_output_0)
  %/layer2/MatMul_output_0 = MatMul(%/activation_fn1/Sigmoid_output_0, %onnx::MatMul_23)
  %/layer2/Add_output_0 = Add(%layer2.bias, %/layer2/MatMul_output_0)
  %/activation_fn2/Sigmoid_output_0 = Sigmoid(%/layer2/Add_output_0)
  %/layer_out/MatMul_output_0 = MatMul(%/activation_fn2/Sigmoid_output_0, %onnx::MatMul_24)
  %/layer_out/Add_output_0 = Add(%layer_out.bias, %/layer_out/MatMul_output_0)
  %output = Softmax[axis = 0](%/layer_out/Add_output_0)
  return %output
}
```

Step 3. Inference MNIST ONNX model with ONNX runtime.

```
# Run model with onnx runtime
import numpy as np
import matplotlib.pyplot as plt
import cv2
import onnxruntime

# Load the ONNX model
session = onnxruntime.InferenceSession('model_mnist.onnx')

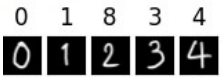

random_array_2d = np.random.rand(1, 28, 28).astype(np.float32)
print(output)

# Test with your own handwritten images.
fig=plt.figure(figsize=(2, 5))
for i in range(1, 11):
    # Load image, Resize to 28x28, Invert and Normalize the pixel values
    path = 'img/' + str(i-1) + '.jpg'
    img_original = cv2.imread(path)
    img = cv2.resize(img_original, (28, 28), interpolation=cv2.INTER_AREA)
    img = cv2.bitwise_not(img)
    img = img / 256
    ax = fig.add_subplot(2, 5, i)
    ax.set_xticks([])
    ax.set_yticks([])
    plt.imshow(img)

    # Convert numpy image into tensor image
    # Select only 1 channel of RGB images is enough.
    # Add dimension to image to have the same format as MNIST data which has 4 dimensions with unsqueeze()
    img = img[:,:,0].astype(np.float32)
    data_input = np.expand_dims(img, axis=0)

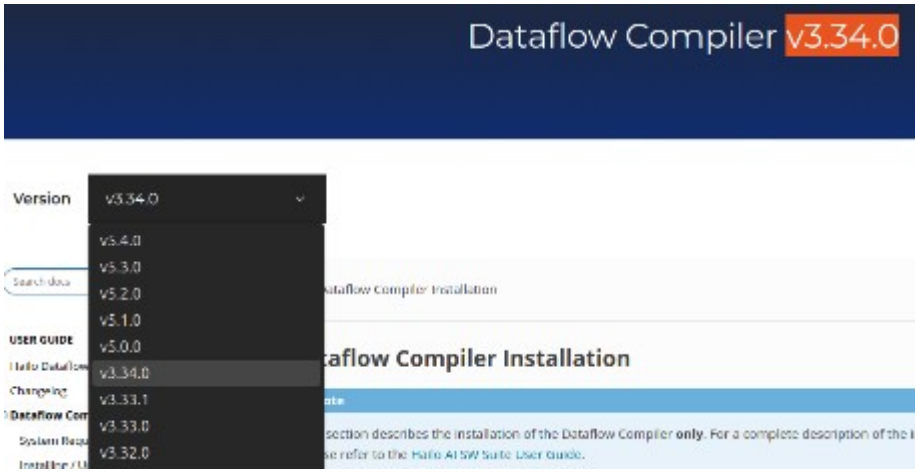
    # Prediction
    predict = session.run(None, {'input': data_input})
    indices_predict = np.argmax(predict)
    print(path, ", predicted number=", int(indices_predict))
    plt.title(str(int(indices_predict)))
```

```
[array([9.4346695e-02, 2.2324764e-05, 1.6347086e-01, 6.2449867e-01,
       7.8253997e-06, 8.4286317e-02, 2.2796572e-04, 4.5060870e-04,
       3.2457624e-02, 2.3119885e-04], dtype=float32)]
img/0.jpg , predicted number= 0
img/1.jpg , predicted number= 1
img/2.jpg , predicted number= 8
img/3.jpg , predicted number= 3
img/4.jpg , predicted number= 4
img/5.jpg , predicted number= 0
img/6.jpg , predicted number= 6
img/7.jpg , predicted number= 7
img/8.jpg , predicted number= 8
img/9.jpg , predicted number= 7
```



Homework

- Register the user account of Hailo and install the **Hailo Dataflow Compiler (DFC) version 3.34.0** on your machine. It will be used in the last lab, using Raspberry Pi + Hailo-8 AI accelerator



- Link for the installation (Must Sign-in first)

https://hailo.ai/developer-zone/documentation/dataflow-compiler-v3-34-0/?sp_referrer=install/install.html

https://hailo.ai/developer-zone/documentation/hailo-sw-suite-2025-10-for-hailo-8-8l/?sp_referrer=suite/suite_overview.html